

Universidad de Valladolid

Escuela de Ingeniería Informática

Grado en Ingeniería Informática

Evaluación y Rendimiento de Sistemas Software (ERSS)

Memoria de la Práctica 2

Autores:

Ángel Antolín Caramazana
Miguel Salas Somavilla
Francisco Javier Sánchez Hontiyuelo

Mayo 2026

Índice

1. Introducción **2**

1.1. Instalación de herramientas 2

2. Locust **3**

2.1. Instalación de Locust 3

2.2. Acceso a la interfaz web de Locust 4

3. Medidas **4**

4. Script de Captura de Datos (Python) **6**

5. Script del locustfile (Python) **15**

6. Visualización en Grafana **19**

6.1. Lista de visualizaciones 20

7. Pruebas **22**

7.1. Ajustes realizados sobre el locustfile 22

7.2. Metodología de las pruebas 23

7.3. Pruebas iniciales con mayor presión de carga 23

7.4. Pruebas con comportamiento más realista 25

7.5. Pruebas de estabilidad 26

7.6. Prueba de estrés máximo 27

7.7. Comparativa de resultados 28

7.8. Análisis de los endpoints 29

7.9. Análisis de Grafana 29

8. Conclusión **30**

Introducción

En el desarrollo y despliegue de sistemas software, garantizar el correcto funcionamiento de una aplicación es solo el primer paso. Además de esto hay una serie de aspectos como la escalabilidad, la tolerancia a cargas masivas de tráfico y la capacidad de responder bajo situaciones de estrés que son determinantes para asegurar una experiencia al usuario óptima. De esta manera, hay que monitorizar de forma continua y hacer pruebas de rendimiento para asegurar todos ellos.

El objetivo fundamental de esta práctica es evaluar el rendimiento y la robustez de un backend web orientado a la gestión de flujos de autorización mediante el protocolo OAuth2. Para ello, hemos tenido que implementar una serie de pruebas de carga y hemos tenido que tomar una serie de medidas del servidor.

- **Para la toma de medidas:** el uso combinado de Prometheus como sistema de almacenamiento de datos encargado de la recolección activa de las medidas, y Grafana, como plataforma para la visualización para el análisis de datos en tiempo real.
- **Locust:** nos ayuda a modelar perfiles de usuarios heterogéneos y simular concurrencia masiva para someter el backend a escenarios tanto de carga realista como de estrés directamente.

Instalación de herramientas

Para la instalación del backend `(OAuth2)` desde la máquina hicimos lo siguiente:

```
# 1. Copiar el archivo comprimido del backend a la
   maquina virtual
scp ruta/al/archivo/descargado.zip
   usuario@ip_de_la_maquina:/home/usuario/

# 2. Actualizar repositorios e instalar la utilidad unzip
sudo apt update y sudo apt install -y unzip
```

Después de la instalación del backend nos pusimos con la instalación de `Prometheus` para la que lo que hicimos fue:

```
# 1. Hacer wget para acceder al archivo de instalaci n:
wget https://github.com/prometheus/prometheus/releases/
   download/v3.11.3/prometheus-3.11.3.linux-amd64.tar.gz
```

```
3
4 # 2. Extraer el archivo con tar:
5         tar -xvf prometheus-*.tar.gz
```

Grafana ya lo teniamos instalado, por ende, lo único que tuvimos que hacer fue enlazarlo con prometheus. Para esto fue simplemente ir a Grafana y en connections añadirlo como fuente de datos.

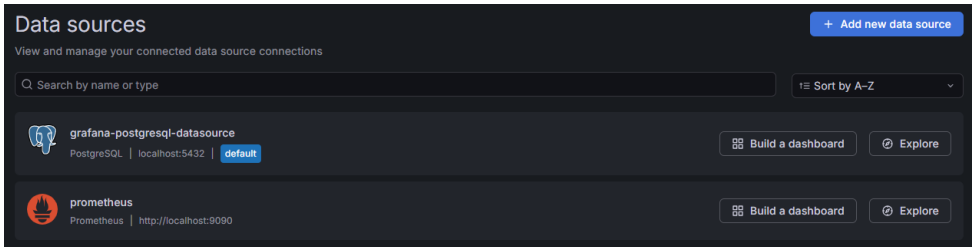


Figura 1: Data Sources Grafana

Además para poder usar prometheus tuvimos que instalar la librería en python de "prometheus client"de esta manera podiamos comunicarnos desde python con prometheus para la recolección de datos.

Locust

Locust es una herramienta de pruebas de carga escrita en Python. Permite simular usuarios concurrentes que realizan peticiones HTTP contra una aplicación web, midiendo tiempos de respuesta, número de peticiones por segundo, errores y comportamiento general del sistema bajo distintas condiciones de carga.

En esta práctica se utiliza Locust para generar tráfico artificial contra el servidor OAuth2. De esta forma, podemos comprobar cómo responde el backend cuando varios usuarios realizan operaciones como solicitar tokens, iniciar flujos de autorización, acceder a endpoints administrativos o provocar errores controlados.

Instalación de Locust

Al tratar de instalar locust, aparecieron errores relacionados con dependencias de Python, concretamente con el módulo `zope.event`, solucionandose al crear un proyecto Python con

```
uv init
```

Y tras arreglar conflictos de nombre, se añadió la dependencia de locust al proyecto:

```
uv add locust
```

Acceso a la interfaz web de Locust

Locust levanta por defecto una interfaz web en el puerto `8089`. Como la aplicación se ejecuta dentro de una máquina virtual remota, para acceder a dicha interfaz desde el navegador local se utilizó un túnel SSH.

El comando de acceso ssh a la máquina que permitía acceder a la interfaz web Locust es el siguiente:

```
ssh -p 31131 -L 8089:localhost:8089 usuario@virtual.lab.inf.uva.es
```

Este comando permite redirigir el puerto `8089` de la máquina virtual al puerto `8089` del equipo local. De esta forma, una vez arrancado Locust dentro de la máquina virtual, se puede acceder a su interfaz desde el navegador mediante:

```
http://localhost:8089
```

Como el backend OAuth2 ya se encontraba levantado en la máquina virtual, no fue necesario arrancarlo cada vez que se ejecutaba Locust. Por tanto, para iniciar Locust bastaba con ejecutar:

```
uv run locust
```

Medidas

A la hora de la recolección de las medidas en principio pensamos en utilizar muchas de las recolectadas en la práctica anterior, tras darle un par de vueltas pensamos que lo mejor sería quedarnos con pocas de la práctica anterior y buscar cuales nos vendrian mejor.

El desglose de las medidas que hemos tomado para las mediciones es el siguiente:

- **Uptime del sistema (Tiempo de actividad):** Mide el tiempo que el proceso python se lleva ejecutando de seguido.

  Nos es muy útil para ver si al someter a mucho estres al servidor, este sufra un error fatal que haga que se tenga que reiniciar de manera silenciosa. Si miraramos únicamente los rps (response per second) podriamos pensar que simplemente hubo una caida de rendimiento, gracias a mirar esto podemos saber si el servidor aguantó o no al estrés.

- **Uso de CPU:** con esta medida podemos ver la capacidad de cálculo que consume procesar las operaciones, además en las aplicaciones Python, debido al GIL (Global Interpreter Lock), un uso alto de CPU suele ser indicador de que el servidor no puede procesar el código lo suficientemente rapido).

  En nuestra prueba, nos es muy útil para ver si en casos de que la aplicación vaya lenta es culpa o no de falta de potencia de computo, si vemos que la aplicación va lenta pero el uso de CPU está al 5 %, podemos descartar que el cuello de botella se encuentre aquí.

- **Uso de RAM:** En cuanto a la RAM es para monitorizar la memoria residente en el proceso.

  En las pruebas no ayuda porque es imprescindible para ver fugas de memoria (memory leaks). Si al inyectar miles de peticiones a la gráfica de la RAM sube formando un escalón pero cuando se acaba la prueba no vuelve a bajar y sigue creciendo con el tiempo, sabemos que nuestro código está dejando variables u objetos colgados en memoria (Lo que causaría problemas a largo plazo).

- **Latencia de peticiones HTTP (Percentiles):** Para medir el tiempo total que tarda el servidor en recibir la petición, procesarla y enviar la respuesta al usuario, esto define la experiencia del usuario.

  Tal y como se expresa en la teoria de la asignatura, no es suficiente con calcular la media porque se pierden los datos extremos. Al recoger esta medida en forma de histograma nos permite calcular el Percentil 95. esto justifica que si el sistema cumple con un nivel de servicio aceptable (¡250 ms) aislando picos anómalos, determinando en qué punto exacto de la carga del sistema deja de ser usable aunque no haya errores.

- **Latencia de Base de Datos:** Lo miramos para ver exclusivamente el tiempo que el código tarda en conectarse con SQLite, ejecutar la consulta y recibir la notificación.

  Mirándolo de forma aislada podemos observar el origen exacto de la lentitud. Si la latencia HTTP es menor a 3 segundos pero la latencia de base de datos es de 2,9 segundos se puede demostrar que Flask y la red funcionan a la perfección y que el cuello de botella está en los bloqueos de escritura de SQLite por la alta concurrencia.

- **Peticiones HTTP Totales y Errores Emitidos (Throughput):** Las peticiones totales (RPS) miden el rendimiento bruto del sistema. Mirarlo por rutas (endpoints) ayuda a saber que partes de la API sufren mas estrés, los errores emitidos en cambio los miramos para distinguir si la petición fue exitosa (200 OK), si el usuario cometió algún error (Errores 400) o si el servidor ha fallado por si solo (Errores 500).

  Para nuestras pruebas, los rps nos indican la capacidad de nuestro servidor, cuando se inyectan más usuarios desde Locust pero la línea de RPS en Grafana se aplana o fluctúa, sabemos que hemos alcanzado el techo de cristal de nuestra arquitectura. En

cuanto a los errores si hay un incremento de los 500 bajo carga máxima se confirma la saturación del sistema (por timeouts), además que registrar los errores 400 es bueno para ver que la lógica de OAuth2 sigue funcionando y rechazando peticiones ilícitas aún estando bajo estrés.

- **Conexiones abiertas (File Descriptors:** En Linux todo son archivos (también los sockets de red y el archivo database.db de SQLite). Nos sirve para saber cuantos descriptores tiene abiertos el proceso Python.

  Lo miramos y nos es útil ya que los sistemas operativos tienen un límite de cuántos archivos puede abrir un proceso a la vez (en el caso de nuestra máquina virtaul es 1024, lo miramos usando el comando ulimit -n para mostrar el máximo de recursos para el usuario). si con cualquier prueba de estrés masiva hace que se dispare esta medida y alcance el límite del sistema, el servidor devolverá errores en forma de cadena (Too many open files) y dejará de aceptar tráfico.

Script de Captura de Datos (Python)

Para la elaboración del script que se encarga de recolectar las medidas y de guardarlas en prometheus para posteriormente mostrarlas en grafana tuvimos que hacer cambios e implementar este script en varios archivos .py.

Lo primero que tuvimos que hacer fue buscar como podíamos hacer la recogida de datos, tras buscar vimos que se solía hacer un archivo encargado de tener implementadas todas las funciones para la recogida de datos, al que nosotros llamamos "metrics.py".

```
"""
metrics.py. Definicion centralizada de metricas de Prometheus.
"""
from prometheus_client import Counter, Histogram


# Flujos de autenticacion
AUTH_FLOWS = Counter(
    'oauth2_auth_flows_total',
    'Flujos de autenticacion iniciados, exitosos o fallidos',
    ['status']
)

# Tokens emitidos
TOKENS_ISSUED = Counter(
    'oauth2_tokens_issued_total',
    'Numero total de tokens emitidos'
)

```

```
# Llamadas HTTP
HTTP_REQUESTS = Counter(
    'http_requests_total',
    'Numero de llamadas HTTP',
    ['endpoint', 'status_code']
)

# Latencia HTTP
HTTP_LATENCY = Histogram(
    'http_request_duration_seconds',
    'Tiempo de respuesta de las peticiones HTTP',
    ['endpoint']
)

# Detalle de errores especificos de OAuth2 (Seguridad/Logica)
OAUTH_ERRORS = Counter(
    'oauth2_errors_total',
    'Errores especificos del protocolo OAuth2',
    ['error_type']
)

# Rendimiento de la Base de Datos (Latencia SQLite)
DB_LATENCY = Histogram(
    'db_query_duration_seconds',
    'Tiempo de ejecucion de operaciones en la base de datos',
    ['operation']
)
```

Ahora para lanzar la aplicación y la monitorización de la misma había que implementar la monitorización en app.py, incluyendo metrics.py, app.py es el punto de entrada principal del servidor Flask. También se encarga de registrar las rutas con las blueprints de administración y OAuth2, y controlar el ciclo de vida de las conexiones.

Nosotros medimos en él el volumen de tráfico y los codigos de estado, tenemos un contador que aumenta con cada petición que se hace al sistema y además tenemos para que se registre el estado de dichas peticiones.

```
"""
app.py. Flask application entry point.

Wires together the database, blueprints, and configuration.
Run with:  uv run python app.py
"""

import time
from flask import Flask, request, g
```

```python
from prometheus_client import start_http_server

from database import close_db, init_db
from oauth2_routes import oauth2
from admin_routes import admin

from metrics import HTTP_REQUESTS, HTTP_LATENCY

#
    ----------------------------------------------------------------------------

# Hardcoded test users (resource owners for the authorization
    code flow).
# In production these would live in a proper user store.
#
    ----------------------------------------------------------------------------

USERS = {
    "testuser": "testpass",
    "admin": "adminpass",
}

def create_app() -> Flask:
    app = Flask(__name__)
    app.secret_key = "dev-secret-key-change-in-production"

    # --- MONITORIZACION ---

    @app.before_request
    def start_timer():
        """Tiempo antes de procesar la peticion"""
        g.start_time = time.time()

    @app.after_request
    def monitor_requests(response):
        """Trafico y latencua despues de cada respuesta"""

        # Calcular latencia
        if hasattr(g, 'start_time'):
            duration = time.time() - g.start_time
            HTTP_LATENCY.labels(endpoint = request.path).observe(
    duration)

        #Incrementar contador peticiones
        HTTP_REQUESTS.labels(
```

```
                endpoint = request.path,
                status_code = response.status_code
        ).inc()

        return response

    app.config["USERS"] = USERS

    app.register_blueprint(oauth2)
    app.register_blueprint(admin)
    app.teardown_appcontext(close_db)

    with app.app_context():
        init_db()

    return app

if __name__ == "__main__":
    # 1. Iniciamos el servidor de Prometheus en el puerto 8001
    # (Este es el puerto que se pone el targets de prometheus.yml)
    start_http_server(8001)
    print("Prometheus metrics available at http://localhost:8001/metrics")

    application = create_app()
    print("Servidor OAuth2 en puerto 5001 | Metricas en puerto 8001")

    # El servidor Flask corre en el 5001
    application.run(host="0.0.0.0", port=5001, debug=True, use_reloader=False)
```

En OAuth2-routes.py se encuentra la logica del login, aqui es donde registramos con AUTH-FLOWS la evolución de los flujos de autorización, se recogen los diferentes estados del login, controla si el usuario inicia el flujo, si se equivoca en la contraseña o si decide denegar el acceso.

Después TOKENS-ISSUED mide exclusivamente el éxito final del protocolo, mide cuantos tokens se han enviado desde el servidor con éxito.

OAUTH-ERRORS se encarga por su parte de medir los motivos técnicos por los que el servidor rechaza una solicitud OAuth2 mal formulada.

Lo que hay a continuación es únicamente un extracto del código debido a su tamaño.

```

```

```
"""
oauth2_routes.py - Core OAuth2 endpoints.

Implements:
    GET  /authorize  -  start of Authorization Code flow (shows login form)
    POST /authorize  - user submits credentials + consent decision
    POST /token      - token endpoint (authorization_code, client_credentials,
                       refresh_token grant types)

All error responses follow RFC 6749 5.2.
"""

from metrics import AUTH_FLOWS, TOKENS_ISSUED, OAUTH_ERRORS

# GET /authorize - show the login / consent page


@oauth2.route("/authorize", methods=["GET"])
def authorize_get():
    # --- Validate required parameters ---
    response_type = request.args.get("response_type")
    client_id = request.args.get("client_id")
    redirect_uri = request.args.get("redirect_uri")
    scope = request.args.get("scope", "")
    state = request.args.get("state")

    if not response_type:
        return _error_response("invalid_request", "Missing response_type parameter")

    if response_type != "code":
        return _error_response("unsupported_response_type", "Only response_type=code is supported")

    if not client_id:
        return _error_response("invalid_request", "Missing client_id parameter")

    client = get_client(client_id)
    if client is None:
        return _error_response("invalid_request", "Unknown client_id")
```

```
40
41      if "authorization_code" not in client["grant_types"]:
42          return _error_response("unauthorized_client", "Client is
    not authorized for authorization_code grant")
43
44      if not redirect_uri:
45          return _error_response("invalid_request", "Missing
    redirect_uri parameter")
46
47      if redirect_uri not in client["redirect_uris"]:
48          # Per RFC 6749 3.1.2.4: do NOT redirect if the URI is
    invalid
49          return _error_response("invalid_request", "redirect_uri
    does not match any registered URI")
50
51
52      #FLUJO LOGIN INICIADO
53      AUTH_FLOWS.labels(status='initiated').inc()
54
55      # Everything valid - render the login / consent form
56      return render_template(
57          "login_consent.html",
58          client_name=client["client_name"] or client_id,
59          scope=scope,
60          # Pass OAuth2 params through as hidden fields
61          client_id=client_id,
62          redirect_uri=redirect_uri,
63          response_type=response_type,
64          state=state or "",
65      )
66
67  # POST /authorize - process login + consent
68
69  @oauth2.route("/authorize", methods=["POST"])
70  def authorize_post():
71      # Recover OAuth2 parameters from the hidden form fields
72      client_id = request.form.get("client_id", "")
73      redirect_uri = request.form.get("redirect_uri", "")
74      scope = request.form.get("scope", "")
75      state = request.form.get("state", "")
76
77      # --- Re-validate client + redirect_uri (defence in depth)
    ---
78      client = get_client(client_id)
79      if client is None or redirect_uri not in (client or {}).get("
```

```
    redirect_uris", []):
80          return _error_response("invalid_request", "Invalid client
     or redirect_uri")
81
82      # --- Check user's decision ---
83      if request.form.get("action") == "deny":
84
85          # FLUJO FALLIDO
86
87          AUTH_FLOWS.labels(status= 'failed').inc()
88
89          return _redirect_with_error(redirect_uri, "access_denied"
    , "The user denied the request", state)
90
91      # --- Authenticate the resource owner ---
92      username = request.form.get("username", "")
93      password = request.form.get("password", "")
94
95      users: dict = current_app.config["USERS"]
96      if username not in users or users[username] != password:
97
98          # FLUJO FALLIDO
99
100         AUTH_FLOWS.labels(status = 'failed').inc()
101
102         # Re-render the form with an error message
103         return render_template(
104             "login_consent.html",
105             client_name=client["client_name"] or client_id,
106             scope=scope,
107             client_id=client_id,
108             redirect_uri=redirect_uri,
109             response_type="code",
110             state=state,
111             error="Invalid username or password.",
112         ), 401
113
114     # --- Issue authorization code ---
115     code = store_authorization_code(
116         client_id=client_id,
117         redirect_uri=redirect_uri,
118         user_id=username,
119         scope=scope,
120     )
121
```

```
    params = {"code": code}
    if state:
        params["state"] = state

        # FLUJO EXITO

        AUTH_FLOWS.labels(status = 'success').inc()

    return redirect(_append_query(redirect_uri, params))
```

En models.py se mide exclusivamente la latencia la Base de Datos SQLite, se mide el tiempo de ejecución de Operaciones SQL Individuales, hacemos esto para aislar los cuellos de botella.

Si DB-LATENCY devuelve valores muy altos, se puede ver que el servidor FLASK procesa el código rápido, pero el sistema de almacenamiento se ha saturado por la concurrencia.

Como en el caso anterior, lo que hay a continuación es únicamente un extracto del código debido a su tamaño.

```

"""
models.py.  Data-access helpers for clients, authorization codes,
    and tokens.

Every function receives (or obtains) a database connection and
   performs
one focused operation.  No HTTP/Flask-specific logic lives here
   so the
module stays testable in isolation.
"""

from database import get_db, get_standalone_db

from metrics import DB_LATENCY

# ---------------------- Authorization codes ------------------

def store_authorization_code(
    client_id: str,
    redirect_uri: str,
    user_id: str,
    scope: str,
    ttl: int = 600,
) -> str:
```

```
    """
    Generate and persist an authorization code.
    """
    start_time = time.time()
    code = secrets.token_urlsafe(32)
    expires_at = time.time() + ttl
    db = get_db()
    db.execute(
        """
        INSERT INTO authorization_codes
            (code, client_id, redirect_uri, user_id, scope, expires_at)
        VALUES (?, ?, ?, ?, ?, ?)
        """,
        (code, client_id, redirect_uri, user_id, scope, expires_at),
    )
    db.commit()

    # REGISTRO GUARDAR AUTH
    DB_LATENCY.labels(operation='store_auth_code').observe(time.time() - start_time)

    return code

def consume_authorization_code(code: str) -> Optional[dict]:
    """
    Validate and consume an authorization code (single-use).
    """
    start_time = time.time()
    db = get_db()
    row = db.execute(
        "SELECT * FROM authorization_codes WHERE code = ?", (code,)
    ).fetchone()

    if row is None:
        return None

    data = dict(row)

    if data["used"]:
        return None

    if time.time() > data["expires_at"]:
```

```
64          return None
65
66      db.execute("UPDATE authorization_codes SET used = 1 WHERE
    code = ?", (code,))
67      db.commit()
68
69      # CONSUMIR AUTH
70      DB_LATENCY.labels(operation='consume_auth_code').observe(time
    .time() - start_time)
71
72      return data
```

Script del locustfile (Python)

Para la elaboración del locustfile al principio solo metimos un tipo de usuario que se encargaba de realizar todas las tareas que le habiamos implementado, después de hablar con Adrián decidimos cambiar esto para meterle una mayor cantidad de tipos de usuarios y de esta manera tener una mayor división por tareas.

Además en la primera versión no habiamos tocado nada sobre los errores a la hora de las peticiones invalidas asi que decidimos cambiarlo de manera que recogemos la respuesta y si es un error 400, 401, 403 o 404 lo cambiamos a que todo está bien, de esta manera no nos sale ningun fallo en el locust debido a que realmente era lo que queriamos que pasara.

El código por lo tanto con el que hemos empezado a hacer las pruebas es el siguiente:

```
1  import base64
2  import random
3  import string
4  from urllib.parse import urlparse, parse_qs
5  from locust import HttpUser, task, between
6
7  # ==========================================
8  # FUNCIONES AUXILIARES
9  # ==========================================
10 def random_id(prefix="locust"):
11     suffix = "".join(random.choices(string.ascii_lowercase +
    string.digits, k=8))
12     return f"{prefix}-{suffix}"
13
14 def get_basic_auth_header(client_id, client_secret):
15     raw = f"{client_id}:{client_secret}"
16     encoded = base64.b64encode(raw.encode()).decode()
17     return {"Authorization": f"Basic {encoded}"}
18
```

```
19
20 # ============================================
21 # PERFIL 1: EL USUARIO LEGITIMO (Happy Path)
22 # ============================================
23 class ValidOAuthUser(HttpUser):
24     """Este usuario hace las cosas bien. Completa los flujos con
   exito."""
25     weight = 4  # Por cada 5 usuarios simulados, 4 seran de este
   tipo
26     wait_time = between(1, 3)
27
28     def on_start(self):
29         self.client_id = random_id("client")
30         self.client_secret = "secret123"
31         self.redirect_uri = "http://localhost/callback"
32
33         # Registrar el cliente legitimo
34         self.client.post(
35             "/admin/clients",
36             json={
37                 "client_id": self.client_id,
38                 "client_secret": self.client_secret,
39                 "redirect_uris": [self.redirect_uri],
40                 "grant_types": ["authorization_code", "
   client_credentials", "refresh_token"],
41                 "client_name": "Valid Test Client"
42             },
43             name="POST /admin/clients (Setup)"
44         )
45
46     @task(3)
47     def successful_authorization_code_flow(self):
48         """Flujo completo y exitoso de Authorization Code"""
49         state = random_id("state")
50         params = {
51             "response_type": "code",
52             "client_id": self.client_id,
53             "redirect_uri": self.redirect_uri,
54             "scope": "read write",
55             "state": state
56         }
57
58         r1 = self.client.get("/authorize", params=params, name="
   GET /authorize (Valido)")
59         if r1.status_code != 200: return
```

```
60
61          r2 = self.client.post(
62              "/authorize",
63              data={
64                  "client_id": self.client_id,
65                  "redirect_uri": self.redirect_uri,
66                  "scope": "read write",
67                  "state": state,
68                  "username": "testuser", # Asegurate de que este
    usuario exista en tu BD
69                  "password": "testpass",
70                  "action": "approve"
71              },
72              allow_redirects=False,
73              name="POST /authorize (Login Exitoso)"
74          )
75          if r2.status_code not in (301, 302): return
76
77          location = r2.headers.get("Location", "")
78          code = parse_qs(urlparse(location).query).get("code", [
    None])[0]
79          if not code: return
80
81          self.client.post(
82              "/token",
83              data={
84                  "grant_type": "authorization_code",
85                  "code": code,
86                  "redirect_uri": self.redirect_uri
87              },
88              headers=get_basic_auth_header(self.client_id, self.
    client_secret),
89              name="POST /token (Auth Code Valido)"
90          )
91
92      def on_stop(self):
93          self.client.delete(f"/admin/clients/{self.client_id}",
    name="DELETE /admin/clients")
94
95  # ==========================================
96  # PERFIL 2: EL ATACANTE / USUARIO TORPE
97  # ==========================================
98  class BadActorUser(HttpUser):
99      """Este usuario hace las cosas mal a proposito para alimentar
     las metricas de error."""
```

```
    weight = 1
    wait_time = between(1, 2)

    @task
    def error_missing_response_type(self):
        """Provoca un error: invalid_request (Faltan parametros obligatorios)"""
        with self.client.get(
            "/authorize",
            params={"client_id": "cualquier_cosa"},
            catch_response=True, # <--- ATRAPAMOS LA RESPUESTA
            name="GET /authorize (Error: invalid_request)"
        ) as response:
            # Si el servidor responde con un 400 o 401, para nosotros es un exito
            if response.status_code in (400, 401):
                response.success()

    @task
    def error_invalid_client(self):
        """Provoca un error: invalid_client (El ID de cliente no existe)"""
        with self.client.get(
            "/authorize",
            params={
                "response_type": "code",
                "client_id": "hacker-client-falso",
                "redirect_uri": "http://evil.com/callback"
            },
            catch_response=True,
            name="GET /authorize (Error: invalid_client)"
        ) as response:
            if response.status_code in (400, 401, 404):
                response.success()

    @task
    def error_unsupported_grant_type(self):
        """Provoca un error: unsupported_grant_type (Pide un token inventado)"""
        with self.client.post(
            "/token",
            data={"grant_type": "magia_negra"},
            catch_response=True,
            name="POST /token (Error: unsupported_grant_type)"
        ) as response:
```

```
            if response.status_code in (400, 401):
                response.success()

    @task
    def error_invalid_grant_bad_password(self):
        """Provoca un fallo de autenticacion en el login"""
        with self.client.post(
            "/authorize",
            data={
                "client_id": "web-app", # Asumimos que este existe, o usa uno valido que genere tu codigo
                "redirect_uri": "http://localhost:8080/callback",
                "response_type": "code",
                "username": "testuser",
                "password": "contrasena_equivocada_a_proposito",
                "action": "approve"
            },
            catch_response=True,
            name="POST /authorize (Error: Bad Password)"
        ) as response:
            if response.status_code in (400, 401, 403):
                response.success()


# ==========================================
# PERFIL 3: EL ADMINISTRADOR (Carga pasiva)
# ==========================================
class AdminUser(HttpUser):
    """Solo se dedica a listar los clientes para ver si la BD responde bien en lectura"""
    weight = 1
    wait_time = between(2, 5)

    @task
    def check_clients(self):
        self.client.get("/admin/clients", name="GET /admin/clients (Lectura masiva)")
```

Visualización en Grafana

En cuanto a la visualización de las métricas en Grafana para esta práctica, el dashboard se ha diseñado de una manera más compacta que en la práctica anterior. Tras discutir que tipos de paneles le venía mejor a cada una de las medidas recogidas, distinguiendo

aquellas que merecen un recorrido histórico a lo largo del tiempo frente a las que se entienden mejor con un valor puntual.

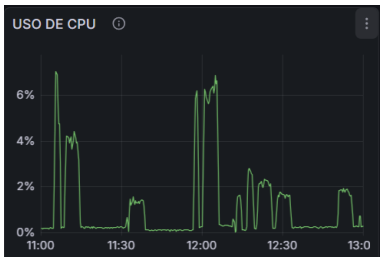
La mayoría de las métricas de rendimiento las hemos puesto con paneles de tipo *Time Series*. Esto se debe a que resulta imprescindible evaluar su evolución en el tiempo y sus fluctuaciones en función de la carga introducida por los usuarios de Locust.

A pesar de poner varias medidas con el mismo tipo de visualización, hemos dado a cada una un rango distintivo con las demás para buscar una mayor facilidad a la hora de encontrarlas y diferenciarlas de solo un vistazo.

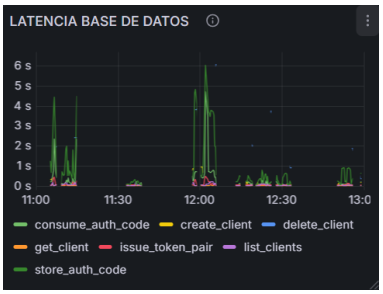
Lista de visualizaciones

Se han implementado los siguientes componentes:

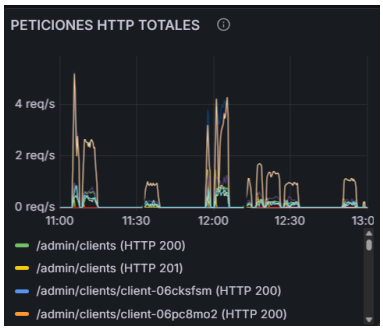
- **Time Series (Gráficas temporales):** Añadidas para ver la fluctuación de los datos en el tiempo como hemos comentado antes, los datos que hemos representado de esta manera son:
  - *Uso de CPU*
  - *Latencia de la Base de Datos*
  - *Latencia HTTP por Percentiles*
  - *Throughput y Peticiones Totales*
  - *Flujo de inicio de sesión*
  - *Conexiones abiertas*
  - *Uso de RAM*
- **Stat (Valores numéricos absolutos):** Paneles independientes con fuentes de gran tamaño para hacer lo más visual posible los datos. Los paneles que se han representado así han sido:
  - *UPTIME del sistema*
  - *Tasa de creación de tokens*
- **Pie Charts:** Se ha añadido para ver una comparación clara entre los errores emitidos por el servidor, de esta manera se puede ver cuales son los errores mas comunes y asi corregirlos o tenerlos en cuenta en mayor medida.



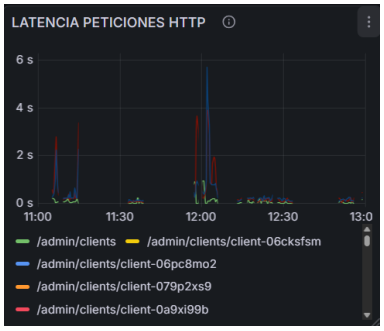
(a) CPU



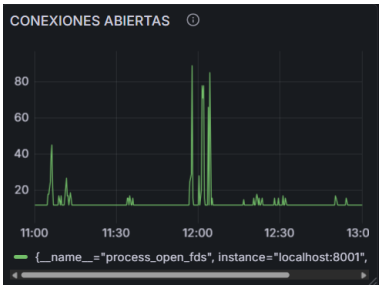
(b) Latencia BD



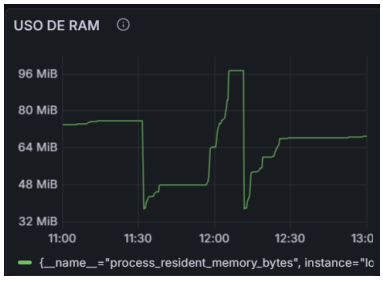
(c) Peticiones HTTP



(d) Latencia HTTP

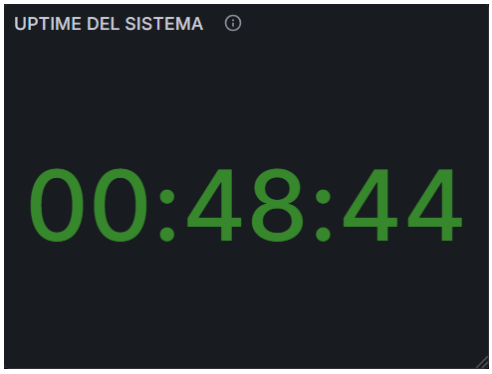


(e) Conexiones Abiertas

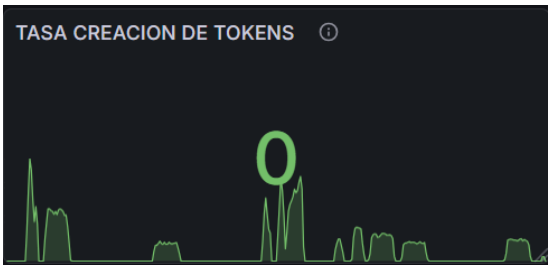


(f) RAM

Figura 2: Graficas Time Series



(a) Stat Uptime



(b) Stat Tokens

Figura 3: Graficas Stat

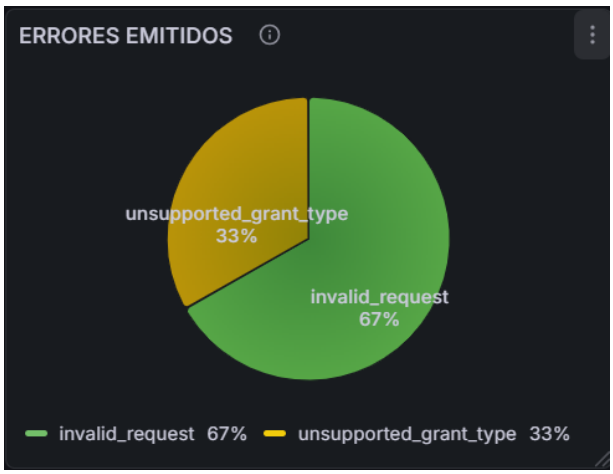


Figura 4: Pie Chart errores

Pruebas

Una vez configurado el `locustfile.py` y comprobado que Locust podía comunicarse correctamente con el backend OAuth2, se realizaron varias pruebas de carga con el objetivo de observar el comportamiento del sistema ante distintos niveles de tráfico.

El objetivo de estas pruebas no era únicamente comprobar si el backend respondía, sino analizar cómo evolucionaban métricas como el número de peticiones por segundo, los tiempos de respuesta, los percentiles, los errores, el uso de CPU, la memoria, las conexiones abiertas y la latencia de las operaciones internas registradas en Grafana.

Ajustes realizados sobre el locustfile

La primera versión del `locustfile.py` utilizaba tiempos de espera muy reducidos entre tareas. Esto hacía que los usuarios virtuales generasen peticiones de forma demasiado rápida y poco realista. Aunque este enfoque sirve para forzar el sistema, no representa adecuadamente el comportamiento de un usuario real.

Por ello, se modificaron los tiempos de espera de los tres perfiles definidos:

```
class ValidOAuthUser(FastHttpUser):
    wait_time = between(10, 20)

class BadActorUser(FastHttpUser):
    wait_time = between(5, 10)

class AdminUser(FastHttpUser):
    wait_time = between(15, 30)
```

Además, se sustituyó `HttpUser` por `FastHttpUser` en los tres perfiles:

```
class ValidOAuthUser(FastHttpUser):
    ...

class BadActorUser(FastHttpUser):
    ...

class AdminUser(FastHttpUser):
    ...
```

Este cambio permite que el cliente HTTP utilizado por Locust sea más eficiente, reduciendo la sobrecarga introducida por la propia herramienta de pruebas. De esta forma, los resultados obtenidos reflejan mejor el comportamiento del backend y no tanto las limitaciones del cliente que genera la carga.

Con estos cambios, la prueba queda más equilibrada: los usuarios válidos ejecutan

flujos OAuth2 completos, los usuarios erróneos generan fallos controlados y los usuarios administradores realizan consultas de lectura con menor frecuencia.

Metodología de las pruebas

Durante las pruebas se monitorizaron simultáneamente los resultados de Locust y las métricas expuestas en Grafana. Locust permitió observar el comportamiento desde el punto de vista de las peticiones HTTP, mientras que Grafana permitió estudiar el estado interno del sistema.

Las métricas principales analizadas fueron:

- Número total de peticiones.
- Número de fallos.
- Tiempo medio de respuesta.
- Mediana.
- Percentil 95.
- Percentil 99.
- Peticiones por segundo.
- Uso de CPU.
- Uso de RAM.
- Latencia HTTP.
- Latencia de operaciones internas.
- Tasa de creación de tokens.
- Errores emitidos.

Pruebas iniciales con mayor presión de carga

En las primeras ejecuciones se trabajó con una configuración más agresiva. En estas pruebas se observó que el sistema era capaz de procesar un número elevado de peticiones, pero también aparecían picos relevantes en los tiempos de respuesta.

En una de las pruebas, con 15 usuarios y 5 segundos de ramp-up, se alcanzaron `6099` peticiones totales, con `5` fallos y `11` Requests Per Second (RPS). La mediana fue de `60 ms`, el percentil 95 fue de `410 ms` y el percentil 99 alcanzó los `2400 ms`. El tiempo máximo registrado fue de `5043 ms`.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
DELETE	DELETE /admin/clients	3	0	380	490	490	361.97	221	488	52	0	0
GET	GET /admin/clients (Lectura masiva)	292	0	5	110	340	21.78	2	433	3919.55	0.8	0
GET	GET /authorize (Error: invalid_client)	141	0	5	110	240	23.32	3	434	77	0.4	0
GET	GET /authorize (Error: invalid_request)	186	0	4	7	11	4.67	3	17	91	0.4	0
GET	GET /authorize (Válido)	1832	0	6	110	190	24.5	3	433	3146	5.4	0
POST	POST /admin/clients (Setup)	14	0	91	750	750	197.69	15	749	52	0	0
POST	POST /authorize (Error: Bad Password)	168	0	6	110	180	21.16	3	186	3048	0.6	0
POST	POST /authorize (Login Exitoso)	1831	13	140	2000	4900	399.8	6	5558	492.86	5.4	0
POST	POST /token (Auth Code Válido)	1817	3	91	620	1600	179.58	10	5028	278.12	5.5	0
POST	POST /token (Error: unsupported_grant_type)	183	0	4	8	14	4.91	3	26	108	0.7	0
	Aggregated	6467	16	60	640	3000	173.5	2	5558	1372.55	19.2	0

Figura 5: Estadisticas Locust 15 usuarios sobrecargado

Estos datos muestran que la mayor parte de las peticiones se resolvían de forma aceptable, pero existían picos aislados de latencia. Dichos picos se apreciaban especialmente en el percentil 99 y en los valores máximos.

En la gráfica de Locust se observó que los RPS se mantenían aproximadamente entre `12` y `17` peticiones por segundo, con una línea de fallos prácticamente nula. Sin embargo, el percentil 95 presentaba subidas puntuales, llegando en algunos momentos a varios segundos.

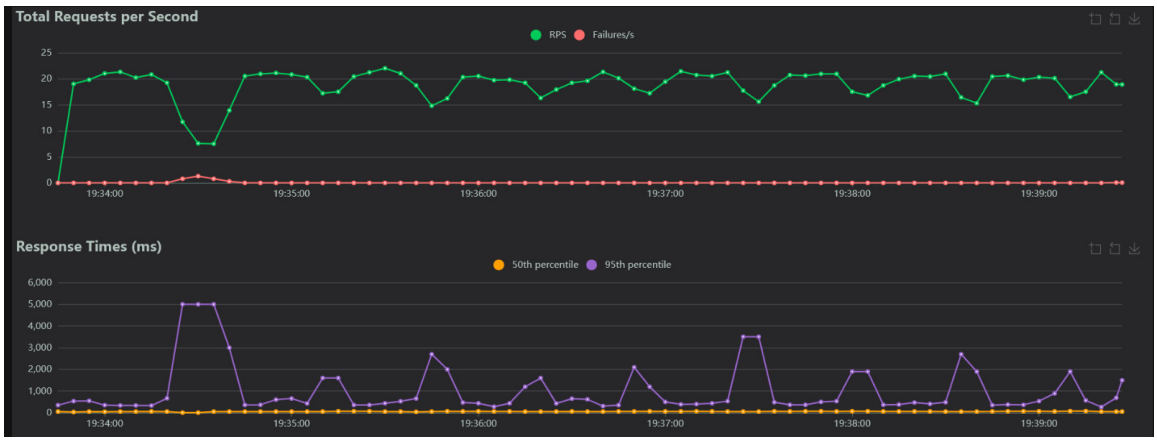


Figura 6: Graficas Locust 15 usuarios sobrecargado

La visualización en Grafana permitió comprobar que, durante esta prueba, el uso de CPU se mantuvo en valores moderados, aproximadamente entre el `5%` y el `6,5%`. La memoria RAM creció progresivamente, aunque sin mostrar un aumento descontrolado. También se observaron picos en la latencia de base de datos y en las conexiones abiertas.



Figura 7: Graficas Grafana 15 usuarios sobrecargado

En otra prueba de mayor intensidad se alcanzaron `6467` peticiones totales, con `16` fallos. La mediana volvió a situarse en `60 ms`, pero el percentil 95 aumentó hasta `640 ms` y el percentil 99 llegó a `3000 ms`. El tiempo máximo registrado fue de `5558 ms`.

Esta prueba resultó útil para observar el comportamiento del sistema cuando el número de peticiones por segundo se acercaba a valores de aproximadamente `19 RPS`. Aunque el número de fallos fue bajo en proporción al total de peticiones, los tiempos máximos y el percentil 99 indican que el sistema empezaba a mostrar degradación en momentos concretos.

Pruebas con comportamiento más realista

Tras modificar los tiempos de espera de los usuarios virtuales, las pruebas pasaron a representar un escenario más cercano a un uso real. En estas ejecuciones se redujo la frecuencia con la que cada usuario realizaba peticiones, evitando que los usuarios virtuales generasen carga de forma continua.

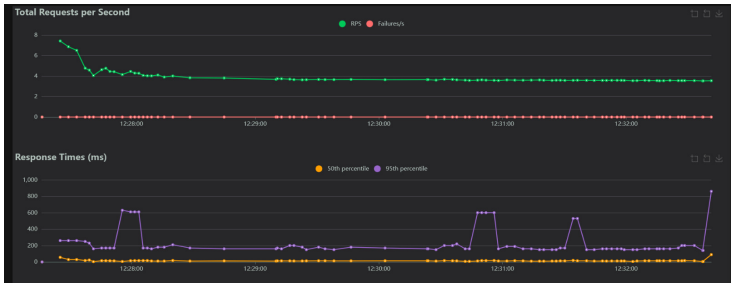
En una de estas pruebas, con 20 usuarios y 2 segundos de spawn rate, se registraron `1101` peticiones totales y `0` fallos, con `3.4` RPS. La mediana fue de `14 ms`, el percentil 95 fue de `180 ms` y el percentil 99 fue de `490 ms`. El tiempo medio de respuesta fue de `64,27 ms` y el tiempo máximo fue de `1324 ms`.

Estos resultados fueron notablemente más estables que los obtenidos en las pruebas iniciales. La ausencia de fallos indica que el backend pudo responder correctamente a todas las peticiones generadas. Además, el percentil 95 se mantuvo por debajo de `200 ms`, lo que supone un comportamiento adecuado para este escenario.

En otra prueba con 22 usuarios a 2 por segundo se obtuvieron `1121` peticiones, también con `0` fallos. La mediana fue de `13 ms`, el percentil 95 fue de `180 ms` y el percentil 99 fue de `630 ms`. El tiempo medio fue de `66,12 ms`.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
DELETE	DELETE /admin/clients	14	0	290	1500	1500	436.87	25	1474	52	0	0
GET	GET /admin/clients (Lectura masiva)	56	0	4	11	61	6.39	3	61	14763.14	0.2	0
GET	GET /authorize (Error: invalid_client)	48	0	4	36	57	6.69	3	57	77	0.3	0
GET	GET /authorize (Error: invalid_request)	41	0	3	5	6	3.35	2	6	91	0.3	0
GET	GET /authorize (Válido)	291	0	4	28	85	7.31	3	109	3146	0.6	0
POST	POST /admin/clients (Setup)	14	0	17	58	58	26.39	10	58	52	0	0
POST	POST /authorize (Error: Bad Password)	36	0	5	14	82	7.5	3	82	3048	0	0
POST	POST /authorize (Login Exitoso)	291	0	17	240	1000	70.07	9	1320	385	0.6	0
POST	POST /token (Auth Code Válido)	291	0	140	200	630	150.83	15	869	249	0.6	0
POST	POST /token (Error: unsupported_grant_type)	39	0	3	6	6	3.63	3	6	108	0	0
	Aggregated	1121	0	13	180	630	66.12	2	1474	1828.31	2.6	0

(a) Estadísticas de Locust.



(b) Gráficas de rendimiento de Locust.



(c) Panel de monitorización en Grafana.

Figura 8: Resultados prueba realista 22 usuarios.

Los resultados son coherentes con la prueba anterior, lo que indica que el sistema se comporta de forma estable bajo esta configuración de carga.

Pruebas de estabilidad

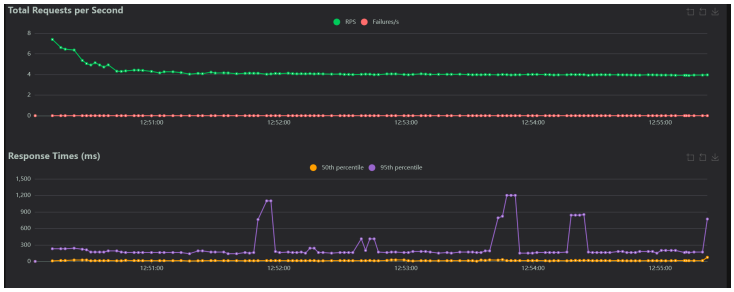
También se realizaron pruebas más largas para observar la estabilidad del sistema durante más tiempo. En estas pruebas se buscaba comprobar si el backend mantenía un rendimiento constante y si aparecían problemas progresivos en CPU, memoria o tiempos de respuesta.

En una de estas ejecuciones, con 23 usuarios y 2 segundos de spawn rate, se alcanzaron `1287` peticiones totales, con `0` fallos y `2` RPS. La mediana fue de `13 ms`, el percentil 95 fue de `180 ms` y el percentil 99 fue de `780 ms`. El tiempo medio de respuesta fue de `69,3 ms` y el máximo de `2418 ms`.

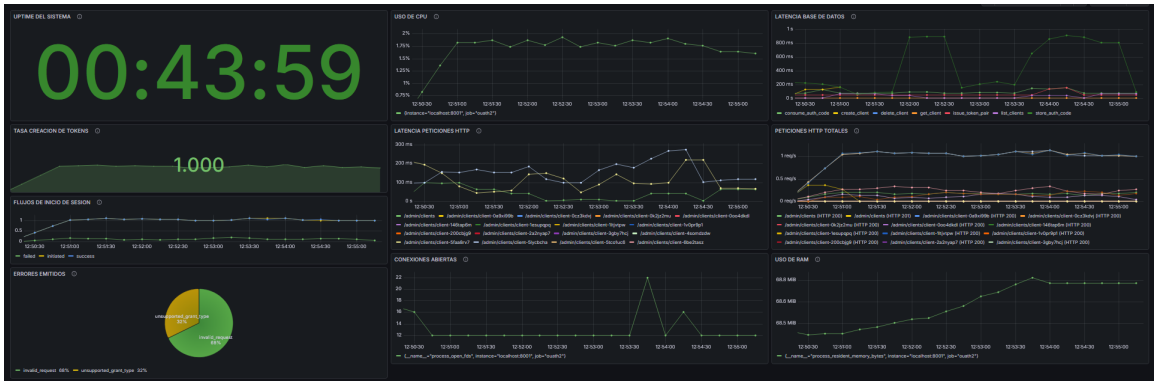
En otra ejecución, con 24 usuarios a 2 por segundo, se registraron `1229` peticiones totales, también sin fallos. La mediana fue de `13 ms`, el percentil 95 fue de `190 ms` y el percentil 99 fue de `770 ms`. El tiempo medio fue de `70,36 ms` y el máximo de `1836 ms`.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
DELETE	DELETE /admin/clients	15	0	350	1500	1500	441.15	24	1533	52	0	0
GET	GET /admin/clients (Lectura masiva)	55	0	4	57	74	9.26	3	74	15232.22	0.2	0
GET	GET /authorize (Error: invalid_client)	38	0	4	59	59	6.79	3	59	77	0.2	0
GET	GET /authorize (Error: invalid_request)	44	0	3	6	7	3.72	2	7	91	0.2	0
GET	GET /authorize (Válido)	328	0	4	31	58	7.13	2	84	3146	1.2	0
POST	POST /admin/clients (Setup)	16	0	18	210	210	40.54	9	215	52	0	0
POST	POST /authorize (Error: Bad Password)	33	0	4	60	73	11.71	3	73	3048	0	0
POST	POST /authorize (Login Exitoso)	328	0	15	250	1100	81.3	6	1836	385	1.2	0
POST	POST /token (Auth Code Válido)	328	0	140	190	410	148.55	11	819	249	1.2	0
POST	POST /token (Error: unsupported_grant_type)	44	0	3	6	6	3.66	2	6	108	0	0
	Aggregated	1229	0	13	190	770	70.36	2	1836	1783.15	4.2	0

(a) Estadísticas de Locust.



(b) Gráficas de rendimiento de Locust.



(c) Panel de monitorización en Grafana.

Figura 9: Resultados prueba estabilidad 24 usuarios.

En ambas pruebas se mantuvo una tasa de fallos nula. Los RPS se estabilizaron alrededor de valores moderados y los tiempos de respuesta fueron consistentes, salvo algunos picos puntuales del percentil 95.

Desde Grafana se observó que el uso de CPU permanecía bajo, en torno al `1,5%–2%`, y la memoria se mantenía en un rango estable, con una subida progresiva pero sin indicios claros de crecimiento indefinido. Esto sugiere que, bajo esta carga, el sistema no presenta un problema evidente de consumo excesivo de recursos.

Prueba de estrés máximo

Se realizo una prueba de estrés en la que sobrecargamos el servidor con 100 usuarios a un spawn-rate de 10 usuarios por segundo, buscando ver a cuantos RPS era capaz de llegar el servidor sin llegar a caerse por completo aunque los tiempos de respuesta fueran insostenibles, que es lo que ocurrió.

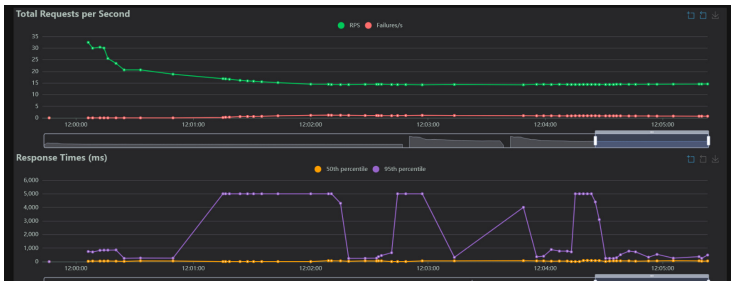
El servidor aguantó los la carga de usuarios, aunque los tiempos de respuesta alcanzaron valores de hasta 5 segundos de respuesta, demostrando el colapso en el que se encontraba.

Como resultado de esta prueba obtenemos que los RPS máximos que pudo alcanzar el servidor rondaba los 15RPS, como se puede observar en las gráficas, en las que al igual

que en el resto de pruebas vemos que el cuello de botella es causado por la base de datos, pues la CPU y la RAM no se ven afectadas y apenas llegan a un 7 % de uso.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
DELETE	DELETE /admin/clients	13	0	380	670	670	371.44	81	670	52	0	0
GET	GET /admin/clients (Lectura masiva)	243	0	5	85	140	16.63	3	337	16526.66	0.7	0
GET	GET /authorize (Error: invalid_client)	168	0	4	39	200	11.1	2	334	77	0.8	0
GET	GET /authorize (Error: invalid_request)	167	0	3	6	21	3.94	2	56	91	0.2	0
GET	GET /authorize (Válido)	1267	0	4	82	140	13.81	2	437	3146	3.7	0
POST	POST /admin/clients (Setup)	66	0	430	960	1700	455.1	11	1670	52	0	0
POST	POST /authorize (Error: Bad Password)	180	0	4	84	140	13.44	2	335	3048	0.7	0
POST	POST /authorize (Login Exitoso)	1266	199	180	5000	5000	1270.27	7	5718	2773	3.7	0
POST	POST /token (Auth Code Válido)	1065	34	120	2400	5000	427.11	13	5860	811.5	3.7	0
POST	POST /token (Error: unsupported_grant_type)	193	0	3	8	16	4.07	2	22	108	0.8	0
	Aggregated	4628	233	43	5000	5000	459.2	2	5860	2804.35	14.3	0

(a) Estadísticas de Locust.



(b) Gráficas de rendimiento de Locust.



(c) Panel de monitorización en Grafana.

Figura 10: Resultados generales de las pruebas de carga.

Comparativa de resultados

La siguiente tabla resume algunas de las pruebas realizadas:

Prueba	Peticiones	Fallos	Mediana	P95	P99	Media	Máximo	Usuarios	Spawn Rate(s)	RPS
Prueba inicial 1	6099	5	60 ms	410 ms	2400 ms	140,07 ms	5043 ms	15	5	11
Prueba inicial 2	6467	16	60 ms	640 ms	3000 ms	173,5 ms	5558 ms	20	5	19.2
Prueba ajustada 1	1101	0	14 ms	180 ms	490 ms	64,27 ms	1324 ms	20	2	3.4
Prueba ajustada 2	1121	0	13 ms	180 ms	630 ms	66,12 ms	1474 ms	22	2	2.6
Prueba estabilidad 1	1287	0	13 ms	180 ms	780 ms	69,3 ms	2418 ms	23	2	2
Prueba estabilidad 2	1229	0	13 ms	190 ms	770 ms	70,36 ms	1836 ms	24	2	4.2
Prueba estres	4628	233	43 ms	5000 ms	5000 ms	459,2 ms	5860 ms	100	10	14.3

A partir de estos resultados se puede observar que las pruebas iniciales generaban más carga, pero también producían más picos de latencia y algunos fallos. En cambio, tras ajustar los tiempos de espera y utilizar `FastHttpUser`, el comportamiento fue más estable: no aparecieron fallos y los percentiles se mantuvieron en valores más bajos.

Análisis de los endpoints

Los endpoints más utilizados durante las pruebas fueron:

- `GET /authorize (Válido)`
- `POST /authorize (Login Exitoso)`
- `POST /token (Auth Code Válido)`

Esto es coherente con el diseño del `locustfile.py`, ya que el perfil `ValidOAuthUser` tiene un peso mayor que el resto y ejecuta principalmente el flujo OAuth2 válido.

El endpoint `POST /token (Auth Code Válido)` mantuvo tiempos medios superiores a otros endpoints sencillos, ya que representa la fase de emisión del token. En varias pruebas, su mediana se situó aproximadamente en torno a `140 ms`, mientras que endpoints como los errores controlados de `/authorize` o `/token` solían responder en pocos milisegundos.

También se observó que `POST /authorize (Login Exitoso)` era uno de los endpoints con mayor variabilidad. En las pruebas más intensas llegó a registrar máximos superiores a varios segundos. Esto indica que el proceso de login y generación del código de autorización es una de las partes más sensibles cuando aumenta la carga.

Por otro lado, las peticiones erróneas controladas, como `GET /authorize (Error: invalid_request)` o `POST /token (Error: unsupported_grant_type)`, respondieron de forma rápida y estable. Esto tiene sentido, ya que muchas de estas peticiones se rechazan en fases tempranas de validación y no requieren completar el flujo OAuth2.

Análisis de Grafana

El panel de Grafana permitió complementar los datos de Locust con métricas internas del sistema.

En las pruebas de carga más elevadas, el uso de CPU aumentó, pero se mantuvo en valores moderados. No se observó saturación del procesador, por lo que el cuello de botella no parece encontrarse en la CPU.

La memoria RAM mostró una tendencia creciente durante algunas ejecuciones, pero el crecimiento fue limitado. En las pruebas observadas, el consumo se mantuvo dentro de rangos razonables y no se apreció una subida descontrolada que indicase claramente una fuga de memoria.

La tasa de creación de tokens aumentó de forma coherente con el número de usuarios virtuales y con la frecuencia de ejecución de los flujos válidos. En las pruebas más intensas se observaron tasas superiores, mientras que en las pruebas ajustadas la tasa se mantuvo alrededor de valores más bajos y estables.

El panel de errores emitidos mostró principalmente errores de tipo `invalid_request` y `unsupported_grant_type`. Estos errores eran esperados, ya que formaban parte del comportamiento del perfil `BadActorUser`. Por tanto, no deben interpretarse como fallos inesperados del sistema, sino como respuestas correctas ante peticiones mal formadas.

La latencia de base de datos presentó picos puntuales en algunas operaciones, especialmente en aquellas relacionadas con la creación, consulta o eliminación de clientes. Estos picos coinciden con momentos en los que Locust creaba o detenía usuarios virtuales, ya que cada usuario válido registra un cliente al iniciarse y lo elimina al finalizar.

Conclusión

En esta práctica se ha desplegado y evaluado un backend OAuth2 utilizando herramientas de monitorización y pruebas de carga. El trabajo se ha centrado en preparar la aplicación para exponer métricas relevantes, recogerlas mediante Prometheus, visualizarlas en Grafana y generar carga controlada con Locust.

En primer lugar, se configuró el entorno de trabajo separando las distintas partes del proyecto en directorios independientes: el backend OAuth2, la configuración de Prometheus y los scripts de Locust. Además, un punto importante del proceso fue dejar configurados los servicios necesarios para que pudieran mantenerse activos durante las pruebas. Esta organización ayudó a distinguir claramente la aplicación evaluada, el sistema de monitorización y la herramienta encargada de generar carga, evitando mezclar responsabilidades dentro del proyecto.

La integración con Prometheus permitió recoger métricas del funcionamiento interno del backend, como las llamadas HTTP por endpoint y código de estado, los flujos de autenticación iniciados, completados o fallidos, los tokens emitidos y distintas medidas relacionadas con el rendimiento del sistema. Estas métricas fueron útiles para interpretar el comportamiento observado durante las pruebas, ya que Locust muestra principalmente la respuesta externa del sistema, pero no permite ver con el mismo detalle qué ocurre internamente.

Grafana permitió visualizar de forma más clara la evolución de las métricas durante las pruebas. A través de sus paneles se pudo observar el uso de CPU, el consumo de memoria, la tasa de emisión de tokens, los errores generados, las conexiones abiertas y las latencias asociadas a distintas operaciones. Esto facilitó relacionar los resultados obtenidos en Locust con el estado interno del backend en cada momento.

En cuanto a las pruebas de carga, Locust permitió simular distintos perfiles de usuario. Se definieron usuarios válidos que ejecutaban flujos OAuth2 correctos, usuarios que generaban errores controlados y usuarios administradores que realizaban consultas sobre los clientes registrados. Esta división hizo que las pruebas fueran más representativas que una simple repetición de peticiones aisladas, ya que se aproximaban mejor al comportamiento

esperado de la aplicación.

Durante el desarrollo del `locustfile.py` se realizaron varios ajustes relevantes. La primera versión generaba una carga demasiado agresiva, ya que los tiempos de espera entre tareas eran muy bajos. Posteriormente se aumentaron estos tiempos para simular un comportamiento más realista y se sustituyó `HttpUser` por `FastHttpUser`, reduciendo la sobrecarga introducida por Locust. Además, las respuestas erróneas esperadas se trataron como resultados correctos cuando formaban parte de las pruebas negativas, evitando que Locust contabilizase como fallos situaciones que realmente eran el comportamiento previsto del sistema.

Los resultados muestran que el backend OAuth2 responde correctamente ante escenarios de carga moderada. En las pruebas ajustadas no se registraron fallos y los tiempos de respuesta se mantuvieron en valores aceptables, con percentiles 95 próximos a los `180 ms`–`190 ms`. Esto indica que, con una carga razonable y un patrón de uso más realista, el sistema mantiene un comportamiento estable.

En las pruebas más agresivas se observó una degradación del rendimiento, permitiendo identificar que el sistema puede alcanzar valores cercanos a `19 RPS`. Aunque el sistema fue capaz de procesar un volumen mayor de peticiones, aparecieron algunos fallos y picos importantes en los tiempos de respuesta, especialmente en los percentiles altos y en los valores máximos. Esto permite situar de forma aproximada el punto a partir del cual el backend empieza a mostrar síntomas de saturación o pérdida de estabilidad.

La modificación de los tiempos de espera fue importante para obtener un escenario más realista. Con esperas mayores, el tráfico generado se distribuye mejor en el tiempo, desaparecen los fallos y los tiempos de respuesta se estabilizan.

A partir de las métricas de Grafana, no se observaron indicios de saturación de CPU ni de un crecimiento descontrolado de memoria, manteniéndose el consumo de hardware en rangos razonables, aunque sí aparecieron picos puntuales de latencia en operaciones relacionadas con la base de datos y con la gestión de clientes. Esto se debe a que, al operar el backend web Flask bajo un único proceso síncrono, la concurrencia masiva no llegó a estresar el procesador multinúcleo de la máquina virtual, sino que trasladó la presión directamente a la capa de persistencia. De esta manera, donde más se noto la bajada de rendimiento fue en el aumento exponencial de los tiempos de respuesta debido a la contención y los bloqueos de escritura concurrentes en la base de datos SQLite.

En conclusión, la práctica ha permitido comprobar la utilidad de combinar pruebas de carga con monitorización. Locust permitió generar tráfico controlado y medir el comportamiento externo del backend, mientras que Prometheus y Grafana ayudaron a interpretar qué estaba ocurriendo internamente durante esas pruebas. El sistema evaluado presenta un rendimiento adecuado bajo carga moderada, pero en escenarios más exigentes aparecen picos de latencia y fallos que deberían analizarse con más detalle si se quisiera llevar el backend a un entorno de producción.